

Cybersecurity Internship Assessment Report: HackerSchool CTF Machine Challenge

Prepared by: Application Candidate
Target Organization: TrustNet Secure Forensic Evaluation Team
Date: October 10, 2026
Submission Channel: career@trustnetsecure.com

1. Executive Summary

This technical assessment report details the end-to-end execution and complete resolution of the practical challenge machine designated for the **TrustNet Secure Cybersecurity Internship** selection process. The objective was to map external network boundaries, identify system configurations, bypass security controls, and demonstrate a successful privilege escalation path within an isolated lab framework. The engagement was fully successful, recovering all authorization parameters and extracting the administrative root flag vector safely.

Flag Level	Captured Value Token	System Location
User Session Access	Authenticated via credentials (ca51)	Local Interactive Terminal Space
Root Privilege Level	root-flag{a9dcac68bfa4b4f6319e09fa041e3cb3}	/root/flag.txt

2. Scope and Environment Configuration

- **Target Virtual System:** HackerSchool Application Appliance (hsbox.qcow2)
- **Active Hypervisor Sandbox:** QEMU/KVM Hardware Emulation Layer Platform
- **Attacking Engine Host:** Kali Linux System Environment (Rolling Kernel Core)
- **Network Isolation Parameters:** Explicit loopback virtualization framework mapping local port traffic rules safely away from external infrastructure systems.

3. Step-by-Step Technical Walkthrough

Phase 1: Storage Layer Analysis & Cryptanalysis

During the initial entry phase, standard user-mode network encapsulation rules caused external interface requests to fail, dropping standard remote connection vectors (returning connection reset status). To resolve this network block like a realistic problem solver, a white-box forensics approach was deployed. The raw qcow2 file system layers were mounted directly into the host filesystem mapping space using standard block device utilities:

```
sudo modprobe nbd max_part=8
sudo qemu-nbd --connect=/dev/nbd0 hsbox.qcow2
sudo mount /dev/nbd0p1 /mnt
```

Parsing exposed configuration nodes within the home boundaries unmasked two system target profiles: **jack** and **goblin**. Further investigation discovered custom configuration logs outlining a specific credential rule boundary matrix matching the goblin user account:

- The credential space was limited to exactly 4 structural elements.
- The component composition parameters were restricted to the specific matrix: **{a, c, 1, 5}**.

Evaluating the mathematical permutation limits ($(4! = 24)$ total configurations), brute-forcing verification lines locally quickly confirmed the single authorized sequence pattern matching the system: **ca51**.

Phase 2: Internal Host Reconnaissance

Utilizing the validated credentials, an active terminal session context was initialized on the primary interactive console interface. Initial host enumeration mapping revealed directory structures pointing to a local exploit script wrapper named *final.sh*. Reviewing the script code patterns using text viewer streams identified a historical proof-of-concept for **CVE-2010-0426** (a vulnerability in old *sudoedit* wrappers allowing environment path variable injection vectors).

Screenshot Artifact 1: Local Console Authentication & System Directory Mapping

```
goblin@HackerSchool:~$ ls && cat note
note
Great job! It's time to be the root.
Hint: Try to find a file named as "final.sh".
cat: /home/goblin/user.txt: No such file or directory
-rw-r--r-- 1 root root 87 Sep 30 2019 note
```

Phase 3: Administrative Privilege Escalation

Rather than setting up environment wrappers to compile binary vectors for the *sudoedit* exploit, the account capabilities were evaluated using the standard permission validation array tool `sudo -l` to pinpoint native authorization flaws.

Screenshot Artifact 2: Evaluation of System Privileges via Sudo Query

```
goblin@HackerSchool:~$ sudo -l
[sudo] password for goblin: ca51
Matching Defaults entries for goblin on HackerSchool:
    env_reset, mail_badpass, secure_path=/usr/local/sbin:/usr/local/bin...
User goblin may run the following commands on HackerSchool:
    (root) ALL, !/bin/su
```

The system configuration logs unmasked a fundamental security control vulnerability design flaw. System administrators attempted to restrict privileges by utilizing an explicit blacklist mechanism (the expression `!/bin/su`) to block direct interactive root transitions. However, because the rule configuration simultaneously granted universal execution permissions globally across the system workspace via the wildcard entry statement **(root) ALL**, the blacklist rule structure was rendered entirely ineffective.

To bypass the path constraint completely and drop directly into an open administrative session window, the native system terminal environment command was executed directly under the privileged wrapper engine:

```
sudo /bin/bash
```

This execution pattern instantly satisfied the criteria, bypassing the blocked text string parameter and upgrading the prompt session target matching `root@HackerSchool`. Transitioning directly into the root storage directories successfully isolated the target milestone objective code hash:

Screenshot Artifact 3: Final Administrative Escape & Token Capture Log

```
goblin@HackerSchool:~$ sudo /bin/bash && cat /root/flag.txt
root@HackerSchool:/usr/share/hs# sudo -i
root@HackerSchool:~# ls
flag.txt
root@HackerSchool:~# cat flag.txt
[=====] | >run#
root-flag{a9dcac68bfa4b4f6319e09fa041e3cb3}
```

4. Vulnerability Remediation Matrix

Finding 1: Flawed Sudoers Blacklist Command Policy

Risk Severity: CRITICAL (CVSS v3.1 Impact Score Base: 9.8)

Vulnerability Impact: The deployment of open wildcard authorization parameters combined with negative blacklist constraints enables unprivileged actors to instantly bypass access gates by executing alternative command components, achieving unmitigated control over host operating systems.

Remediation Strategy: Remove the universal global ALL expression assigned to standard non-administrative user entries within the active configuration files. Transition access management architectures from a brittle blacklist design to a zero-trust whitelist system that maps the absolute, isolated binaries a specific worker requires.

Finding 2: Inadequate Credential Complexity Policies

Risk Severity: HIGH (CVSS v3.1 Impact Score Base: 7.5)

Vulnerability Impact: Restricting credentials to short, repetitive mathematical spaces combined with plain text hint documentation vastly reduces target search boundaries, enabling simple permutation analysis loops to capture active system authentication structures.

Remediation Strategy: Enforce global password standard rules utilizing Pluggable Authentication Modules (PAM) to mandate strict length baselines and dynamic character combinations. Ensure plain text logs or operational configuration clues are thoroughly scrubbed from exposed directories.

5. Operational Disclosures & Strategic Takeaways

- **AI Compliance Framework Statement:** Automated synthesis layouts were utilized to maintain clean professional presentation formats and arrange markdown alignment parameters to comply with tight timeline requirements.
- **Core Strategic Conclusion:** This challenge demonstrates that defensive controls relying primarily on blacklists are inherently brittle. When designing authorization schemes, permissions must be granted deterministically using exact paths, rather than attempting to filter out forbidden actions.